Table of Contents

Spike Report: local model selection under a 6 GB VRAM ceiling

Field	Value
Document ID	NOVA-SPK-001
Version	1.0
Status	Partial
Owner	Laxmi Poudel
Date	2026-09-07
Question	Can a local model hold NOVA's test plan JSON schema, on this hardware, at usable latency?
Answer	Yes for the 4B-parameter class. 12 of 12 schema-conformant, fully GPU-resident, 9 to 26 s warm. The 8B class is unmeasured
Decisions affected	DL-021, DL-022, DL-023, DL-024, DL-025

Why this ran first

Planning had "discrete GPU, 8 GB+ VRAM" written down as a confirmed input, and the candidate model list was built around 9B models. If no local model could hold the schema at tolerable latency, the whole workflow decision would have had to reopen. So this was the cheapest way to test the assumption everything else rests on.

Good instinct, because the assumption was wrong.

Environment

GPU	NVIDIA GeForce RTX 4050 Laptop
VRAM	6141 MiB (6 GB), not 8 GB+
Driver	610.62
OS	Windows 11 Home 10.0.26200
Runtime	Ollama 0.33.3, host install, API on `127.0.0.1:11434`
Python	3.12.10

Norton Security Suite holds three `model_host.exe` processes on the GPU, so usable VRAM is below the nominal 6141 MiB. Recorded so measurements taken here are comparable elsewhere.

Method

Twenty synthetic requirements, five per playbook type (API/CRUD endpoint, auth/permission flow, UI form, data migration), each with four acceptance criteria written to be machine-checkable. All synthetic, so it's committable, and it doubles as the seed for the evaluation golden dataset.

A Pydantic `TestPlan` model defines the test-plan v1 schema. `TestPlan.model_json_schema()` goes straight into Ollama's `format` parameter, which applies constrained decoding, and the same model validates the response. That's deliberate: it tests whether one Pydantic definition can serve as request validation, generation schema, and persistence type at once.

Settings: `temperature=0`, `seed=42`, `num_ctx=8192`, thinking disabled.

The prompt uses the intended production shape. Trusted system template carrying the playbook and its required case types, requirement wrapped in `<requirement>` tags and explicitly labelled as untrusted data that must not be followed as instructions.

Deterministic scoring per generation:

- Schema validity, first attempt
- AC coverage: fraction of the requirement's ACs referenced by at least one case
- Hallucinated ACs: references to AC ids that don't exist
- Required case-type presence: fraction of the playbook's mandated types that appear

The scoring functions have an assertion-based self-test, which passes.

Results

This was a smoke run at n=4 per model, one requirement per playbook type. The numbers are directionally useful and statistically weak. Percentile latency at n=4 is basically min/max and shouldn't be quoted as p50/p95.

Model	Resident @ 8K ctx	Processor	Schema valid	AC coverage	Required types	Warm latency	Cases/plan
`qwen3:4b`	3.9 GB	100% GPU	4/4	0.875	1.000	12.6 to 17.4 s	4 to 6
`gemma3:4b`	2.9 GB	100% GPU	4/4	1.000	1.000	18.0 to 21.0 s	5 to 6
`phi4-mini`	3.6 GB	100% GPU	4/4	1.000	0.917	9.4 to 25.7 s	3 to 5
`qwen3:8b`			not measured				
`granite3.3:8b`			not measured				

Raw per-requirement output: `artifacts/run-log-2026-09-07.txt`.

Three individual results worth noting:

- `gemma3:4b` took 115.1 s on its first call. That's cold model load, not generation. Warm calls for the same model ran 18 to 21 s. Cold load is a real UX and evaluation-runtime cost and has to be excluded from latency metrics or reported separately.
- `qwen3:4b` covered only 2 of 4 acceptance criteria on the delete-user requirement, its one weak result. Every other generation across all three models covered all four.
- `phi4-mini` missed a required case type once, on the auth flow, at 2 of 3 types present.

Findings

The hardware assumption was wrong. 6 GB, not 8 GB+. The original 9B candidate list isn't viable, since a 9B at Q4 leaves effectively no KV-cache headroom on this card.

Constrained decoding held everywhere. 12 of 12 generations valid first attempt, no repair retry needed. Ollama accepted the Pydantic-generated JSON Schema including its nested `$defs`/`$ref` structure, so the single-source-of-truth property the design leans on is real.

4B-class fits comfortably. 2.9 to 3.9 GB resident at 8K context with 100% GPU offload, leaving headroom on a 6 GB card. Context budget isn't the binding constraint I expected it to be at this model size.

Warm latency is much better than the target. The draft NFR was p95 â‰¤90 s end to end. Generation alone came in at 9 to 26 s warm. Even adding retrieval, a critic pass, and persistence, there's a lot of margin. That target should be tightened once the full pipeline exists, because one with this much slack doesn't constrain anything.

Hybrid reasoning models need thinking turned off. qwen3 is one. Left at default, thinking tokens dominate generation time and interact badly with constrained decoding. Required config, not an optimization.

Content quality is the discriminator, not structure. Since structural validity was 100% everywhere, model selection comes down to AC coverage, required case-type adherence, and latency. That's a much harder thing to measure, and it's what the evaluation harness is for.

Limitations

These numbers will get quoted later, so:

1. n=4 per model. Way too small for percentile latency, or for separating models whose quality scores are close.
2. Two of five candidates never measured. Both 8B ones are on disk, but the run stopped before reaching them. The 8B fit question is open.
3. One hardware config, one run, no repeats. Run-to-run variance is unmeasured, and variance is what evaluation thresholds eventually have to come from.
4. AC coverage trusts the model's own `covers_acceptance_criteria` labelling. A case can claim an AC without meaningfully covering it, so the metric is an upper bound.
5. Required case-type presence trusts the model's own `case_type` tagging, same problem.
6. No duplicate-case detection. That needs embeddings, which aren't chosen yet.
7. Cold-load latency contaminates any naive percentile, and I only caught it because one outlier was big enough to notice.

Where this leaves the model choice

Default to a 4B-class model. On what's measured, `gemma3:4b` leads on content quality with perfect AC coverage and required case-type presence across all four requirements, at 2.9 GB resident, the smallest footprint of the three. `phi4-mini` is the latency leader but missed a required case type. `qwen3:4b` is the most consistent on speed but had the weakest coverage result.

This is provisional, not a decision. Four requirements per model can't separate them. It needs confirming against the full twenty, and ideally against repeat runs, before it goes into an ADR as settled.

Follow-up

#	Action	Why
1	Run all 20 requirements against all 5 models	Turns a provisional lean into an actual decision
2	Measure `qwen3:8b` and `granite3.3:8b` residency and offload split	Determines whether 8B is viable on 6 GB at all, or only with CPU offload
3	Repeat runs at fixed seed to measure	Evaluation thresholds have to come from measured

#	Action	Why
	variance	variance
4	Separate cold load from warm generation everywhere	Otherwise every percentile is contaminated
5	Spike the embedding model and fix its dimension	Sticky. The dimension goes into the schema, and changing it later forces a re-embedding migration
6	Hand-audit case-type and AC labels on a sample of plans	Establishes how far the two deterministic metrics can be trusted
7	Verify host Ollama is reachable from inside a container	Known first-run friction for anyone cloning the repo

Reproducing it

Everything is in `artifacts/`: the requirement set, the harness, the raw log.

```
py -3 -m pip install pydantic requests
```

```
set NOVA_LIMIT=4 && py -3 artifacts/spike.py gemma3:4b phi4-mini qwen3:4b
```

Drop `NOVA_LIMIT` for the full twenty. `py -3 artifacts/spike.py --selftest` runs the scoring self-test.